

Server API Routes

Server API Routes, also known as Resource Routes, are the standard way to fetch and mutate data in Storefront Next. They implement the Backend-For-Frontend (BFF) pattern—your client-side components communicate with these server endpoints, which securely handle B2C Commerce API calls, authentication, and sensitive business logic.

How It Works

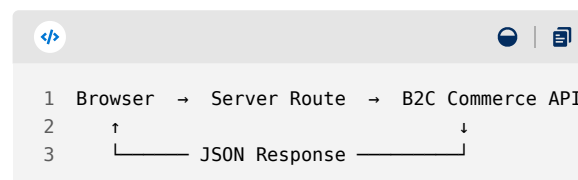
All data fetching routes through server endpoints. Server routes handle:

- API authentication: SLAS flows, client secrets, and token management.
- B2C Commerce API proxying: All SCAPI calls route through server endpoints.
- Callbacks and redirects: Auth redirects, webhooks, JWKS proxying.
- Cookie and header management: Session handling, locale preferences.
- JSON responses: Data for `useFetcher()` or client-side `fetch()` calls.

This approach keeps credentials secure, reduces client bundle size, and centralizes data access patterns.

The Backend-For-Frontend Pattern

Server routes implement the Backend-for-Frontend (BFF) pattern. Your storefront app acts as an intermediary between the client and backend services.



This architecture provides:

- API aggregation: Combine multiple Commerce API calls into single endpoints.
- Security: Keep API credentials and sensitive logic on the server.

Contents

How It Works

The Backend-For-Frontend Pattern

Server Route Architecture

Server-Side Data Layer

Route Types

Request and Response Patterns

Error Handling

Call Server Routes from UI Code

Security Best Practices

File Naming Conventions

Dependencies

See Also



Server Route Architecture

Storefront Next uses React Router 7 in framework mode. Routes are defined by files in `src/routes/` by using the flat routes convention.

A route module becomes a **resource route** (API endpoint) when it exports only data functions such as `loader` and `action`, without a default React component.

Key Concepts

Term	Definition
Server route	Any route file in <code>src/routes/</code> that runs on the server.
Resource route	A server route that exports only data functions (no default component).
Loader	A function that handles GET requests and returns data.
Action	A function that handles mutations (POST, PUT, DELETE).

Naming Conventions

Storefront Next uses these naming conventions.

- `resource.*` files map to `/resource/...` endpoints.
- `action.*` files map to `/action/...` endpoints.
- `loader.*` files map to `/loader/...` endpoints.
- `_empty.*` files bypass parent layouts. Use this prefix when a route needs to render without the standard page wrapper, for example, the logout action that redirects immediately.

Server-Side Data Layer

All data fetching in Storefront Next happens on the server. Resource routes use `loader` and `action` functions that run server-side for both initial page loads and client-side navigation.

This server-side approach provides:

- Security: Sensitive API keys and business logic stay on the server.
- Performance: Reduced client bundle size.

**Important**

Always use server-side `loader` and `action` functions. While React Router supports `clientLoader` and `clientAction`, Storefront Next enforces a server-only data retrieval. Client-side loaders expose your app to security risks and violates the architecture. If you have a rare edge case that seems to require client-side data fetching, consult with the team before implementing.

Route Types

Server routes support three main patterns: loaders for GET requests, actions for mutations, and dynamic parameters for flexible endpoints.

Loader Routes (GET)

Use a `loader` to respond to GET requests. This example shows a store search endpoint.

```
1 // src/extensions/store-locator/routes/resource
2 import { data, type LoaderFunctionArgs } from
3 import { extractResponseError } from '@lib/ut
4 import { createApiClient } from '@lib/api-cl
5
6 export async function loader({ request, contex
7   try {
8     const url = new URL(request.url)
9     const postalCode = url.searchParams.ge
10    const maxDistance = url.searchParams.g
11    const distanceUnit = url.searchParams.
12
13    const clients = createApiClient(contex
14
15    const { data: stores } = await clients
16      params: {
17        query: {
18          postalCode,
19          maxDistance: maxDistance ?
20          distanceUnit: distanceUnit
21        },
22      },
23    })
24
25    return Response.json({ success: true,
26  } catch (error) {
27    const { responseMessage, status_code }
28    return data(
29      { success: false, error: responseM
30      { status: Number(status_code) }
31    )
32  }
33 }
```



```
1 // src/routes/action.set-locale.ts
2 import { data, type ActionFunction } from 'react-router-dom'
3 import { localeCookie } from '@middlewares/cookie'
4
5 export const action: ActionFunction = async ({
6   request,
7   response,
8 }) => {
9   if (!locale) {
10    throw new Response('Locale is required')
11   }
12
13   const cookieHeader = await localeCookie.set(locale)
14
15   return data(
16     { success: true },
17     {
18       headers: {
19         'Set-Cookie': cookieHeader,
20       },
21     )
22   }
23 }
```

Dynamic Route Parameters

Use dynamic segments when a family of endpoints shares a common shape.

```
1 // src/routes/resource.auth.$operation.ts
2 import type { ActionFunctionArgs } from 'react-router-dom'
3
4 type AuthHandler = (request: Request, context: Context) => Promise<Response>
5
6 const authHandlers: Record<string, AuthHandler> = {
7   'refresh-token': handleRefreshToken,
8   'login-guest': handleLoginGuest,
9   'login-registered': handleLoginRegistered,
10 };
11
12 export async function action({ request, params }) {
13   const operation = params.operation as string
14
15   const handler = authHandlers[operation];
16   if (!handler) {
17     return Response.json({
18       success: false,
19       error: `Unknown auth operation: ${operation}`
20     });
21   }
22
23   try {
24     const result = await handler(request, context);
25     return Response.json({ success: true, data: result });
26   } catch (error) {
27     return Response.json({ success: false, error });
28   }
29 }
```



/resource/auth/login-guest.

Calling B2C Commerce APIs with useScapiFetcher

For standard B2C Commerce operations, use the `useScapiFetcher` hook. This hook pairs with a generic server route so that you can call any SCAPI method from your components while keeping the actual API communication on the server.

You don't interact with the underlying server route directly—the hook handles everything for you.

```
1 import { useScapiFetcher } from "@hooks/use-s
2 import { useEffect } from "react";
3
4 export function ProductDetails({ productId }:
5   // The hook expects: (clientName, methodName)
6   // Options use the SCAPI structure: { params
7   const fetcher = useScapiFetcher("shopperProc
8     params: {
9       path: { productId },
10    },
11  });
12
13  useEffect(() => {
14    fetcher.load();
15  }, [productId]);
16
17  if (fetcher.state === "loading") return <div>
18  if (!fetcher.success) return <div>Error: {fe
19  if (!fetcher.data) return null;
20
21  return (
22    <div>
23      <h1>{fetcher.data.name}</h1>
24      <span>{fetcher.data.price}</span>
25    </div>
26  );
27 }
```

For mutations, use the `submit` method.

```
1 import { useScapiFetcher } from "@hooks/use-s
2
3 export function UpdateProfile({ customerId }:
4   const fetcher = useScapiFetcher("shopperCust
5     params: {
6       path: { customerId },
7     },
8     body: {},
9   });
10
11  const handleSubmit = (formData: FormData) =>
12    fetcher.submit(formData);
13  };
14
```



```
20     }  
21   >  
22     <input name="firstName" placeholder="First Name" type="text" value={data.firstName} />  
23     <input name="lastName" placeholder="Last Name" type="text" value={data.lastName} />  
24     <button type="submit" disabled={fetcher.loading || fetcher.submitting}>  
25       {fetcher.state === "submitting" ? "Saving" : "Save"}  
26     </button>  
27   </form>  
28   );  
29 }
```

Note

While `useScapiFetcher` looks like a direct API call, it serializes your request, sends it to the server route, executes the SDK method on the server, and returns the result. The hook provides these convenience properties: `data` (the response payload), `errors` (any error messages), `success` (a boolean), and `state` (loading, submitting, or idle).

Tip

Don't await the `load()` or `submit()` methods expecting data to be populated when the promise resolves. These methods trigger the fetch but return immediately. Use the `state` property or `useEffect` to react to data changes instead.

Request and Response Patterns

Server routes use standard Web APIs, such as [Request](#), [Response](#), and [FormData](#), for handling requests and responses. For detailed patterns, see the [React Router Actions](#).

Error Handling

Handle errors gracefully and return appropriate status codes. Don't expose sensitive information in error messages.

```
</> | [Icons]  
  
1 import type { ActionFunctionArgs } from "react-router-dom";  
2 import { data } from "react-router";  
3 import { extractResponseError } from "@lib/ut";  
4  
5 export async function action({ request }: ActionFunctionArgs) {  
6   try {  
7     const formData = await request.formData();  
8     // ...  
9   } catch (error) {  
10    // ...  
11  }  
12 }
```



```
13   const { responseMessage, status_code } = response;
14
15   return data({ success: false, error: responseMessage });
16 }
17 }
```

Status Code Guidelines

Status	Use Case
400	Validation errors for missing or invalid input.
401	Authentication required.
403	Authorization denied.
404	Unknown operations or resources.
500	Unexpected server failures.

Call Server Routes from UI Code

Use React Router's `useFetcher()` hook to call server routes without triggering a full page navigation. For complete documentation, see [React Router's Data Loading](#).

```
1 import { useFetcher } from "react-router";
2
3 function LocaleSwitcher() {
4   const fetcher = useFetcher();
5
6   return (
7     <fetcher.Form method="post" action="/action" >
8       <select name="locale">
9         <option value="en-US">English</option>
10        <option value="fr-FR">Français</option>
11      </select>
12      <button type="submit">Change Language</button>
13    </fetcher.Form>
14  );
15 }
```

Tip

Don't await the `fetcher.load()` or `fetcher.submit()` methods expecting `fetcher.data` to be populated when the promise resolves. These methods trigger the request but return immediately. Use



Security Best Practices

Server routes handle sensitive operations. Follow these practices.

Validate Input

Always validate and sanitize input from requests.

```
1 export async function action({ request }: Acti
2   const formData = await request.formData();
3   const email = formData.get("email");
4
5   if (!email || typeof email !== "string") {
6     return Response.json({ error: "Valid email
7   }
8
9   // Proceed with validated input
10 }
```

Keep Secrets Server-Side

Never embed client secrets in browser code. Server routes exist specifically to keep credentials secure.

```
1 // ✅ Correct: Secret stays on server
2 export async function action({ context }: Acti
3   const clientSecret = process.env.COMMERCE_AF
4   // Use secret in server-side API call
5 }
```

Avoid Open Proxies

Use the generic Commerce SDK proxy
(`resource.api.client.$resource.ts`) carefully. If you implement similar patterns, enforce an allowlist of methods and validate parameters. Don't expose a "call-anything" proxy without guardrails.

Set Cache Headers Intentionally

Public endpoints must set appropriate `Cache-Control` headers.

```
1 return data(
2   { products },
3   {
4     headers: {
5       "Cache-Control": "public, max-age=300",
6     },
7   },
8 );
```


[Try for free](#)

File Naming Conventions

File Name	Route URL
<code>resource.stores.ts</code>	<code>/resource/st</code>
<code>resource.auth.\$operation.ts</code>	<code>/resource/au</code>
<code>resource.api.client.\$resource.ts</code>	<code>/resource/ap</code>
<code>action.set-locale.ts</code>	<code>/action/set-</code>
<code>action.set-currency.ts</code>	<code>/action/set-</code>
<code>action.place-order.ts</code>	<code>/action/plac</code>
<code>loader.wishlist-products.tsx</code>	<code>/loader/wish</code>
<code>oauth2.jwks.ts</code>	<code>/oauth2/jwks</code>
<code>_empty.logout.ts</code>	<code>/logout</code>

Dependencies

Package	Purpose
<code>react-router</code>	Framework with loader/action pattern
<code>@salesforce/storefront-next-runtime</code>	Runtime utilities, SCAPI types

See Also

- [Salesforce B2C Commerce API \(SCAPI\) Client](#)

DID THIS ARTICLE SOLVE YOUR
ISSUE?

[Share your feedback](#)

[Try for free](#)

DEVELOPER CENTERS

Heroku
MuleSoft
Tableau
Commerce Cloud
Lightning Design System
Einstein
Quip

POPULAR RESOURCES

Documentation
Component Library
APIs
Trailhead
Sample Apps
AppExchange

COMMUNITY

Trailblazer Community
Events and Calendar
Partner Community
Blog
Salesforce Admins
Salesforce Architects

© Copyright 2026 Salesforce, Inc. [All rights reserved](#). Various trademarks held by their respective owners. Salesforce, Inc.
Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#) [Terms of Service](#) [Privacy Information](#) [Responsible Disclosure](#) [Trust](#) [Contact](#) [Use of Cookies](#)

[Cookie Preferences](#)  [Your Privacy Choices](#)

